Python for computational Problem Solving · Unit_02 · Unit 2: Collections & Functions

DRAFT — UE26CS151A · Unit_02 · Unit 2: Collections & Functions · Lists and Tuples

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · Lists and Tuples
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

Lists and Tuples

1 Lists and Tuples

Study focus: Lists and Tuples. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000002 · EV0000003

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Linear data structure where elements have a linear ordering and list has got some of the characteristics.

Key details

- a straightforward task, you would read three variables A B C and then each one of them have been assigned some value and then you will perform A plus B plus C and at the end you're going to be getting the sum.
- If I ask you to perform the sum of thousand numbers, the sum of thousand numbers.
- In this case, what you can do is very simple, where you can iterate over the loop or run over the loop a thousand times, and then try to find out the sum by updating the sum O variable.
- If we ask you to arrange these thousand elements in an ascending order.
- Fine, arranging the thousand elements in an ascending order.
- In this case, it becomes impossible for us to make use of thousand different variables and then perform the arranging operation.
- So in such case, the main reason is you need all the thousand items to be there at a same point of time because you are trying to do the comparison.
- it is always useful to have all the different elements under a similar name.
- Elements in a list are heterogeneous in nature, can be heterogeneous in nature.
- It could be either homogeneous or heterogeneous.
- The moment we say it as homogeneous, it is of the similar type, and when it is an heterogeneous, it could be of different types.
- Lists can be accessed using indexing operation.
- Lists can be.
- Elements of a list can be accessed using indexing operation.

- Request you to see to the video on indexing operation.
- Indexing is also called as a subscript.
- And indexing always starts with zero.
- indexing always starts with zero and this that's the reason because of this particular reason it is also called as zero based indexing.
- And followed by a list can grow or shrink in the sense you once the list is created you can add or also you can remove the elements.
- List is an iterable.
- In this case it is eager.
- I the list is eager in nature.
- I can have a nested list, a list within one list.
- And I can assign one list to the another list.
- In such case both the list would be pointing to the same list.
- I can also create a sublist by making use of an operation called as a slicing operation.
- Please also look into the video of slicing operations.
- And Python provides you so many number of built-in functions where you can manipulate the list, you can work with the list, you can modify the lists.
- Before me going and executing the things, let me quickly iterate over the PPTs that are there.
- I'm going to be demonstrating it to you, so do not worry.
- And as I said you, there are so many operations that are available for us in order to perform the operations which we are going to be also seeing.
- Other operations also include concatenation, sorting.

Example or application from the lecture

- Before we discuss to find out the characteristics of a list, let's try to find out why do I really need a list.
- Okay, we are gonna be looking into each one of these attributes with respect to a particular example.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000002 · EV0000003 · EV0000004 · +1 source units in pack.json

Core explanation · 05:00–09:59

Let's say L S T is equal and twenty thirty forty fifty.

Key details

- Linear ordering means.
- Just look at this and say print list.
- I'm trying to do the same thing here.
- I can I'm just getting my execution part ready.
- I will do all of them at the end I will run so I will say print of what is the length of the list.
- This is the first operation that I want to do.
- List provides there are some common functionalities that we can do with the list.

- Explain it to you in an easier way.
- And then again I will come back as and when it is required for us.
- I will say list is equal to ten comma, twenty comma, thirty, forty and fifty.
- Only difference between interactive and your what is that you have to give state.
- The length of the list is five in this case.
- It tries to find out what is the minimum element that is present.
- I can also perform something called as a sum operation on the list.
- it which adds every element in the list and gives you.
- List plus list.
- In this case, plus is called as a concatenation operation.
- Concatenation operation merges two lists and returns you a single list.

Example or application from the lecture

- When I say linear ordering, my input, the way in which the input is given and the way in which the output is given is same.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000006 · EV0000007 · EV0000008 · +1 source units in pack.json

Core explanation · 10:00–14:59

concatenation operation merges two lists and it returns you a single list.

Key details

- And there is something also called as a repetition operation.
- That's called as a repetition operation.
- Multiply a list and number of times.
- Values have been repeated.
- List elements can be duplicate in nature.
- And see I should have taught you how to create a list.
- Nevertheless, we will do it now.
- Creating an empty list is with the help of an open bracket and a close bracket.
- L S T is equal to L I S T of this.
- And I'll see what is the type of list.
- It is still a list itself.
- In this case, list of this is called as a constructor.
- It's called as a constructor.
- And in this case, the list is called as an empty list.
- How do you create a list?
- Make use of an open and closed brackets in order to represent.
- I can also create a list with the elements where I have already demonstrated.
- And let me s check with the list.
- At this point list is empty.
- This is my list.
- What and all I can do with this particular list.

- As list you can see there are n number of elements, but they are all belong to or they all have a similar name called as LST.
- But LST has a so many number of elements that are present.
- and access every element present in a list.
- How will I access the elements present in the list?
- I can make use of an indexing operation.
- As mentioned earlier, please look into my indexing video.
- as indexing starts from zero, it is also called as what?
- Zero based indexing.
- Called as zero based indexing.
- This is fine.
- Lists also supports slicing operation.
- List also supports a slicing operation.
- Again, see to the video on slicing.
- I can just mention from one and I will not mention anything to stop.
- This is a start value and this is a stop value.
- Okay, so I will say one to end which means start from a index one and go till the index end which is 50 in this case.
- This is fine.
- From three, start from one and stop at four.
- The moment I give four, which means four minus one because one less than the existing one.
- That from one go till three and one and three the values that I'm telling are the indexed values.
- They are the indexed values.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000010 · EV0000011 · EV0000012 · +1 source units in pack.json

Core explanation · 15:00–19:59

And I will try to find out what is the length of my list.

Key details

- Length of my list is five.
- All the values of a list are of same type.
- And then followed by a string.
- In this case list L one is called as heterogeneous in nature.
- The reason is it has an integer type, it has a floating type, and it also has a string type.
- Typo mistink in terms of the string, never mind, do not worry about it.
- Hope it is clear to you.
- How will I do it?
- I'll say DIR of a list to find out what functions does the list have.
- one more thing.
- If I want to find out or access fifty, what do I have to do?
- I will say list of minus one.

- there are many things that I've worked on the indexing topic.
- Please look into the video.
- Do with a few of the operations quickly.
- When I say append 100, it always appends the value to the end.
- You can see that the value hundred is present.
- I can also insert an item at a particular position.
- First parameter is the location that you need to specify and the second parameter is the value of which you want to insert.
- List is this and as I mentioned zero is now being replaced by two hundred.
- So hopefully you have understood the append method followed by the insert method.
- See, it's popped out the last element.
- Again, pop out one more, so it will pop out fifteen and so on.

Example or application from the lecture

- And if you can see the example says that a list has some integer values.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000014 · EV0000015 · EV0000016 · +1 source units in pack.json

Core explanation · 20:00–24:59

If nothing it represents it removes an item from the left side right to left, and if I say pop without any argument, it always removes the last item.

Key details

- And if it is pop with a particular argument.
- In this case the argument is representing the index value.
- I will remove the value hundred.
- One second, just wait.
- Previously I had removed the value 100 and I am trying to remove it.
- 10 is also not there.
- And when you remove the elements gets removed.
- As previously seen, if you're trying to remove a value which is not present, then it throws you an error.
- I'll say remove thousands.
- Okay, I cannot remove because the value thousand is not part of the list.
- List X does not contain this particular Y.
- I can also do this.
- This is my new list.
- List is 20 present in list?
- Answer is yes.
- Is 200 present in list?
- Is 200 not in?
- 200 not in the list.
- Yes, it is still true because it is not there.

- Index value we have already seen.
- Did we check with the index value?
- Just give me time.
- No, pop append insert.
- I will also try to add few values to the list.
- List.append of four hundred.
- I can also do list dot index of twenty.
- What is the index value of 20?
- 20 is present at the zeroth location.
- Two hundred and five, some number, some random number.
- Is two not five present in the list?
- No, it is not present, so it says it is an error.
- I'll say list dot count.
- How many times a particular element is present?
- If something is not present from where it can give you.
- And I can also say list.sort.
- There are some programming examples that we have to work with respect to the lists which I'm going to be doing in your coming classes.

Example or application from the lecture

- Let's look into what is our lists.
- Suppose if I say.
- For example, 200.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000018 · EV0000019 · EV0000020 · +1 source units in pack.json

Core explanation · 25:00–29:59

Hope you have understood.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000022 · EV0000023 · EV0000024 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Lists and Tuples transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/029_Lists_and_Tuples__c3dbcf69ee3f547f.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Lists and Tuples transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/029_Lists_and_Tuples__c3dbcf69ee3f547f.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Lists and Tuples transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/029_Lists_and_Tuples__c3dbcf69ee3f547f.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
Lists and Tuples transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/029_Lists_and_Tuples__c3dbcf69ee3f547f.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. No historical archive question page met the unit-match candidate threshold; all subject-level and ambiguous pages remain in datasets/question-archive.json.
2. 3 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.

6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00002	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare...
SRC00003	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00004	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare...
SRC00005	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare...

C Revision and retrieval practice